Chapter 6

Multithreading with POSIX Threads

6.1 Learning Objectives and Scope

After this chapter, students should be able to create and join POSIX threads, give each worker stable arguments, prove that a block partition covers an array exactly once, combine per-thread results, implement a bounded queue with condition variables, and shut a worker down without losing queued items. Chapter 5 established why shared data needs synchronization; here we turn those rules into complete C programs. Chapter 7 supplies shorter OpenMP expressions of some of these operations. General algorithm design and formal performance analysis appear in Chapters 8 and 10.

6.2 The Pthreads Programming Model

The POSIX Threads (Pthreads) API gives a C program explicit control over worker creation, termination, and coordination. A thread starts in a function of type `void *(*)(void *)`. Its single pointer argument can refer to a structure holding several inputs. `pthread_create` writes a thread identifier, and `pthread_join` waits for one joinable thread to finish. Each function returns zero on success and an error number otherwise; these error numbers are returned directly, rather than through `errno`.

Table 6.1: The small Pthreads API used in this chapter

Call	Role	Rule in our examples
`pthread_create`	Start a worker	Argument storage stays alive while it runs.
`pthread_join`	Wait for completion	Join before reading a worker's result.
Mutex lock / unlock	Protect queue state	Hold the same mutex while testing or changing its fields.
Condition wait	Sleep until state may have changed	Recheck the predicate in a `while` loop.
Signal / broadcast	Notify waiter(s)	Change the protected predicate before notifying.

The start routine may return a pointer result, but passing a pointer to a per-worker record is often clearer: the worker writes its own record, and the creator reads it after joining. The record and any pointed-to input must outlive the worker. In the programs below, both are local variables in `main`; `main` does not return until all workers have joined.

6.2.1 Joinable and detached threads

New threads are joinable by default. A successful join reclaims the thread's termination resources, and a thread must be joined at most once. A detached thread releases these resources on termination but cannot be joined. Detachment is useful when no caller needs a completion result and some other protocol keeps every referenced object alive. Both chapter programs use joins because their results and cleanup depend on knowing that workers are finished. Neither calling `pthread_create` nor seeing the worker print proves that its work has finished.

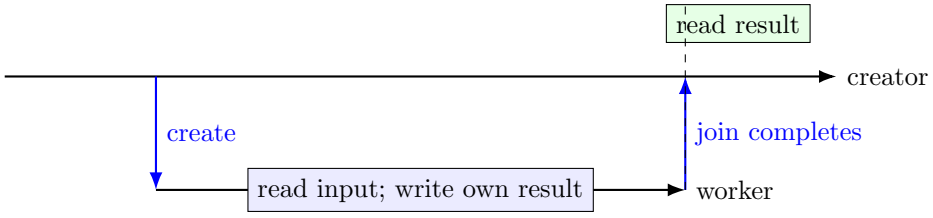


Figure 6.1: The creator may read a worker's result after a successful join. Positions show ordering, not elapsed time.

6.3 Safe Arguments and Ownership

A common failed pattern passes `&t`, the address of a loop index, to each worker. Every worker then points at the *same* changing object. A worker may observe a later value, and unsynchronized access to the index is a data race. Passing the integer as a cast pointer is also nonportable. Give each worker its own structure instead:

Listing 6.1: One stable record per worker

```
typedef struct {
    const int *data;
    size_t begin, end;
    long long partial;
} Work;

Work work[WORKERS];
pthread_t threads[WORKERS];
/* Fill work[t] before creating worker t. */
pthread_create(&threads[t], NULL, sum_range, &work[t]);
/* Read work[t].partial only after joining worker t. */
```

The input array is shared but read-only while workers run. Worker t alone writes `work[t].partial`. Distinct records mean no two workers update the same result object. This is an ownership argument, not an assumption about lucky scheduling. A worker must not retain a pointer to a block-scope object after that object's lifetime ends; joining before `main` returns satisfies this requirement here.

6.4 Block Partitioning with a Remainder

Let N be the number of elements and $P > 0$ the number of workers. Write $N = qP + r$, where $q = \lfloor N/P \rfloor$ and $0 \le r < P$. Give the first r workers $q + 1$ elements and the others q elements. For worker t, with $0 \le t < P$, use half-open bounds

$$b_t = tq + \min(t, r), \qquad e_t = (t+1)q + \min(t+1, r).$$

Thus $b_0 = 0$, $e_{P-1} = N$, and $e_t = b_{t+1}$. Adjacent ranges touch but never overlap. Their lengths differ by at most one. The half-open convention also makes an empty range legal when $P > N$.

Table 6.2: Worked partition of $N = 1003$ elements among $P = 4$ workers: $q = 250$, $r = 3$

Worker t	b_t	e_t	Elements
0	0	251	251
1	251	502	251
2	502	753	251
3	753	1003	250

This quotient-and-remainder formula avoids computing tN, which can overflow an integer type for very large arrays. Another valid formula uses $\lfloor tN/P \rfloor$ and $\lfloor (t+1)N/P \rfloor$; it may place the longer blocks with different workers. The correctness condition is the same: cover $[0, N)$ once with disjoint ranges. Cyclic assignments may help when item costs vary greatly, but this chapter's uniform addition favors contiguous blocks and simple ownership. Chapter 8 treats general load balancing.

6.5 A Complete Parallel Sum

The first runnable program adds the integers 1 through 1003. Each worker accumulates into a local variable, writes one partial result, and returns. The main thread joins every worker, combines the partials, and checks the arithmetic-series formula $N(N+1)/2 = 503{,}506$. No lock is needed for the input or the distinct output records.

From the project root, compile and run:

Listing 6.2: Build and run the block-partitioned sum

```
cc -std=c11 -O2 -Wall -Wextra -pthread \
  examples/chapter6/parallel_sum.c -o parallel_sum
./parallel_sum
```

The full source is `examples/chapter6/parallel_sum.c`. Its worker and setup are:

Listing 6.3: Parallel sum: worker, records, and thread creation

```
/* Compile: cc -std=c11 -O2 -Wall -Wextra -pthread parallel_sum.c -o parallel_sum */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { N = 1003, WORKERS = 4 };

typedef struct {
    const int *data; /* Shared, read-only after thread creation. */
    size_t begin, end; /* This worker owns [begin, end). */
    long long partial; /* Only this worker writes its own result. */
} Work;

static void *sum_range(void *argument)
{
    Work *work = argument;
    long long sum = 0;
    for (size_t i = work->begin; i < work->end; ++i)
        sum += work->data[i];
```

```
    work->partial = sum;
    return NULL;
}

int main(void)
{
    int data[N];
    pthread_t threads[WORKERS];
    Work work[WORKERS];

    for (size_t i = 0; i < N; ++i)
        data[i] = (int)i + 1;

    for (size_t t = 0; t < WORKERS; ++t) {
        /* Quotient/remainder gives balanced, disjoint blocks without t*N. */
        size_t q = N / WORKERS, r = N % WORKERS;
        work[t] = (Work){data, t*q + (t < r ? t : r),
                         (t+1)*q + (t+1 < r ? t+1 : r), 0};
        int rc = pthread_create(&threads[t], NULL, sum_range, &work[t]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            return EXIT_FAILURE;
        }
    }
```

Listing 6.4: Parallel sum: joining, combining, and checking

```
    long long total = 0;
    for (size_t t = 0; t < WORKERS; ++t) {
        int rc = pthread_join(threads[t], NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(rc));
            return EXIT_FAILURE;
        }
        total += work[t].partial; /* Join makes the worker's write available. */
        printf("worker %zu: [%zu,%zu) partial=%lld\n",
               t, work[t].begin, work[t].end, work[t].partial);
    }
    printf("total=%lld expected=%lld\n", total,
           (long long)N * (N + 1) / 2);
    return total == (long long)N * (N + 1) / 2 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

The printed worker lines follow the order of the *join loop*, so they are deterministic even though the workers may run in any order. The final line is `total=503506 expected=503506`. The independent formula catches missing or duplicated work; equality alone would not prove that every input was visited, but it is a useful check alongside the range proof above. For a general input whose sum may overflow `long long`, a production program must select a wider representation or detect overflow.

6.5.1 Why the join matters

The worker's assignment to `partial` occurs before it returns. After `pthread_join` succeeds, the main thread can read that result. Reading it immediately after `pthread_create` would be premature. The join also lets `main` keep `data` and `work` alive until the worker is done. A failed create or join is reported; the program exits rather than silently printing a partial answer. In a

long-running application, a create failure would need a cleanup path that joins already-started threads before freeing their input.

6.6 Waiting for a Predicate

When a consumer needs an item that has not arrived, repeatedly checking an empty queue wastes CPU time. A condition variable puts the consumer to sleep. Its meaning is deliberately modest: a notification says the protected state *may* now satisfy the predicate. It does not hand ownership of an item to the waiting thread. A condition wait atomically releases the associated mutex while sleeping and reacquires it before returning. The waiter therefore tests the predicate again while holding that mutex.

Listing 6.5: Canonical wait on protected queue state

```
pthread_mutex_lock(&queue.mutex);
while (queue.count == 0 && !queue.closed)
    pthread_cond_wait(&queue.not_empty, &queue.mutex);
/* The mutex is held here; inspect count and closed again. */
pthread_mutex_unlock(&queue.mutex);
```

The `while` is required even with a single consumer: POSIX permits a wait to return without the predicate being true. With multiple consumers, another consumer may also take the item before this one reacquires the mutex. A signal sent while no thread waits is not stored for later; the durable information is the queue state under its mutex. Signalling after changing that state prevents a waiting consumer from sleeping past work that is already available.

6.7 A Bounded Queue with Clean Shutdown

The second complete program has one producer, one consumer, and a circular queue of capacity three. The producer inserts 1 through 12 and then marks the queue closed. The consumer keeps taking items until the queue is *both closed and empty*. It sums the items and reports 78. This extends Chapter 5's queue invariant into a functioning protocol.

Table 6.3: Queue state and the action permitted under the mutex

State	Producer	Consumer
`count == 0`, open	May insert	Waits on `not_empty`
`0 < count < CAPACITY`	May insert	May remove
`count == CAPACITY`	Waits on `not_full`	May remove
Closed, `count > 0`	Finished	Drains remaining items
Closed, `count == 0`	Finished	Returns end-of-stream

The mutex protects `head`, `tail`, `count`, `closed`, and the occupied entries. It preserves $0 \le$ `count` $\le$ `CAPACITY` and prevents an entry from being read before insertion or overwritten before removal. The producer waits on `not_full` only while the queue is full. The consumer waits on `not_empty` only while the queue is empty and open. Closing wakes consumers so that an empty queue cannot leave them asleep forever.

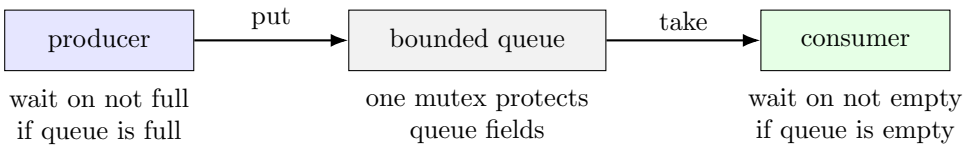


Figure 6.2: Two conditions describe different predicates on one mutex-protected queue. Arrows show the direction of item movement.

Build and run from the project root:

Listing 6.6: Build and run the condition-variable queue

```
cc -std=c11 -O2 -Wall -Wextra -pthread \
  examples/chapter6/condition_queue.c -o condition_queue
./condition_queue
```

The full source is `examples/chapter6/condition_queue.c`. These excerpts show the queue operations and shutdown path; the source file also contains the complete type, initialization, and `main` routine.

Listing 6.7: Queue insertion and removal under one mutex

```

/* Only the producer calls put. The mutex protects all queue fields. */
static void put(int value)
{
    pthread_mutex_lock(&queue.mutex);
    while (queue.count == CAPACITY)
        pthread_cond_wait(&queue.not_full, &queue.mutex);
    queue.values[queue.tail] = value;
    queue.tail = (queue.tail + 1) % CAPACITY;
    ++queue.count;
    pthread_cond_signal(&queue.not_empty);
    pthread_mutex_unlock(&queue.mutex);
}

/* Return 0 only when closed AND drained; output is set only on success. */
static int take(int *output)
{
    pthread_mutex_lock(&queue.mutex);
    while (queue.count == 0 && !queue.closed)
        pthread_cond_wait(&queue.not_empty, &queue.mutex);
    if (queue.count == 0) {
        pthread_mutex_unlock(&queue.mutex);
        return 0;
    }
    *output = queue.values[queue.head];
    queue.head = (queue.head + 1) % CAPACITY;
    --queue.count;
    pthread_cond_signal(&queue.not_full);
    pthread_mutex_unlock(&queue.mutex);
    return 1;
}
```

Listing 6.8: Queue closure, consumer, and thread joins

```c
static void *produce(void *unused)
{
    (void)unused;
    for (int value = 1; value <= ITEMS; ++value)
        put(value);
    pthread_mutex_lock(&queue.mutex);
    queue.closed = 1;
    pthread_cond_broadcast(&queue.not_empty);
    pthread_mutex_unlock(&queue.mutex);
    return NULL;
}

static void *consume(void *result)
{
    long long *sum = result;
    int value;
    while (take(&value))
        *sum += value; /* Only this consumer writes sum. */
    return NULL;
}

int main(void)
{
    pthread_t producer, consumer;
    long long sum = 0;
    int rc = pthread_create(&consumer, NULL, consume, &sum);
    if (rc != 0) { fprintf(stderr, "consumer create failed: %d\n", rc); return 1; }
    rc = pthread_create(&producer, NULL, produce, NULL);
    if (rc != 0) { fprintf(stderr, "producer create failed: %d\n", rc); return 1; }
    rc = pthread_join(producer, NULL);
    if (rc != 0) { fprintf(stderr, "producer join failed: %d\n", rc); return 1; }
    rc = pthread_join(consumer, NULL);
    if (rc != 0) { fprintf(stderr, "consumer join failed: %d\n", rc); return 1; }
    printf("sum=%lld expected=%d\n", sum, ITEMS * (ITEMS + 1) / 2);
    pthread_cond_destroy(&queue.not_empty);
    pthread_cond_destroy(&queue.not_full);
    pthread_mutex_destroy(&queue.mutex);
    return sum == ITEMS * (ITEMS + 1) / 2 ? 0 : 1;
}
```

Closing is a state transition, not a special data item. If the queue still contains values, the consumer drains them before returning zero from `take`. The producer uses `pthread_cond_broadcast` at closure because more than one consumer might be waiting in an extended version. This example has only one producer; multiple producers would need a rule for which one closes the queue after all have finished. The short example checks creation and join failures but omits repetitive checks on mutex and condition-variable calls; real applications should handle those errors and define a cleanup policy. Destroy the synchronization objects only after all threads have joined.

6.8 Thread Pools and Task Granularity

A persistent set of workers can repeatedly take jobs from a queue. This is a *thread pool*: threads are created once, while tasks can be submitted many times. The bounded queue above is the essential handoff mechanism, but a full pool would also represent each task, return its result,

support multiple producers, and define what happens to queued tasks at shutdown. Those extra lifecycle rules matter more than the name of the pattern.

One thread per tiny task may spend a substantial fraction of its time creating and joining threads. A fixed pool amortizes that cost but adds queue coordination. For a fixed-size array sum, long-lived workers or a single round of Pthreads are enough; a pool offers little advantage. This chapter identifies the engineering tradeoff, while Chapter 10 explains how to measure it fairly. Likewise, binding threads to cores or tuning NUMA placement needs hardware-aware evidence from Chapter 2 and measured results rather than a universal rule.

6.9 Common Failure Modes

Table 6.4: Diagnose the protocol before changing the schedule

Symptom	Likely mistake	Repair
Wrong worker IDs	Passed the same loop index address	Give each worker a stable record.
Wrong final sum	Read before join or overlapped ranges	Join, then check every bound.
Consumer hangs	Closed without waking waiter	Broadcast after closing.
Empty after wake	Guarded wait with `if`	Recheck with `while`.
Thread resources linger	Join was omitted	Join or deliberately detach.

6.10 Summary

Pthreads makes thread lifetime and shared-state protocols explicit. Stable per-worker records, disjoint ranges, private partials, and joins make the sum program easy to reason about. A condition variable waits for a predicate on mutex-protected state; closure plus notification lets a consumer drain a queue and terminate. These mechanisms are the concrete foundation for the OpenMP worksharing constructs in the next chapter.

6.11 Exercises

1. For $N = 17$ and $P = 5$, compute q, r, and every half-open range. Explain why the ranges cover exactly 17 elements.

2. Change the sum program to eight workers. Predict the final total and identify the workers, if any, that receive empty ranges when N is changed to five.

3. Explain why reading `work[t].partial` before joining thread t is unsafe even when the worker's arithmetic is correct.

4. Modify `condition_queue.c` to use two consumers, each with its own result record. Specify how to combine their totals and how both discover closure.

5. Suppose a producer marks the queue closed while two items remain. Trace the next three calls to `take`. State the returned values and the final return status.

6. Design a shutdown rule for a queue with three producers. State who closes the queue, when, and which thread joins which workers.

7. Describe a pool task record for evaluating a function on one input. Explain who owns the record before submission, while queued, and after completion.

6.12 Selected Worked Answers

1. $q = 3$, $r = 2$. The ranges are $[0, 4)$, $[4, 8)$, $[8, 11)$, $[11, 14)$, and $[14, 17)$. Their lengths sum to $4 + 4 + 3 + 3 + 3 = 17$ and adjacent endpoints match.

2. The total remains 503,506. With $N = 5$ and $P = 8$, workers 5, 6, and 7 receive empty ranges; workers 0 through 4 each receive one element.

3. The worker may still be writing its result, so the main thread would read the same ordinary object without the required ordering. A successful join establishes completion before the read.

4. Give each consumer a separate `long long` result, join both, then add their results. The producer broadcasts after setting `closed`; each consumer drains available work and exits only when closed and empty.

5. The next two calls return the remaining items successfully, in FIFO order. The third returns zero because the queue is both closed and empty.

6. The main thread can create all three producers, join all three, then lock the queue, set `closed`, broadcast to waiting consumers, and unlock. It then joins the consumers. No producer can insert after closure because all have already joined.

7. A task record can contain the function pointer, input pointer, result location, and completion state. The submitter owns it until the pool accepts it; the queue or worker owns it until completion; the submitter may reclaim it only after a completion signal or join-like protocol. The input and result storage must remain alive throughout.

API reference: The Open Group, *POSIX.1-2024*, Base Specifications, pthread functions: `https://pubs.opengroup.org/onlinepubs/9799919799/functions/`.